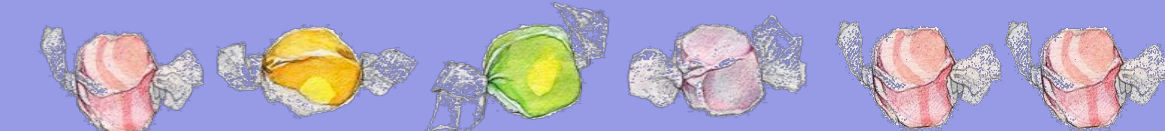
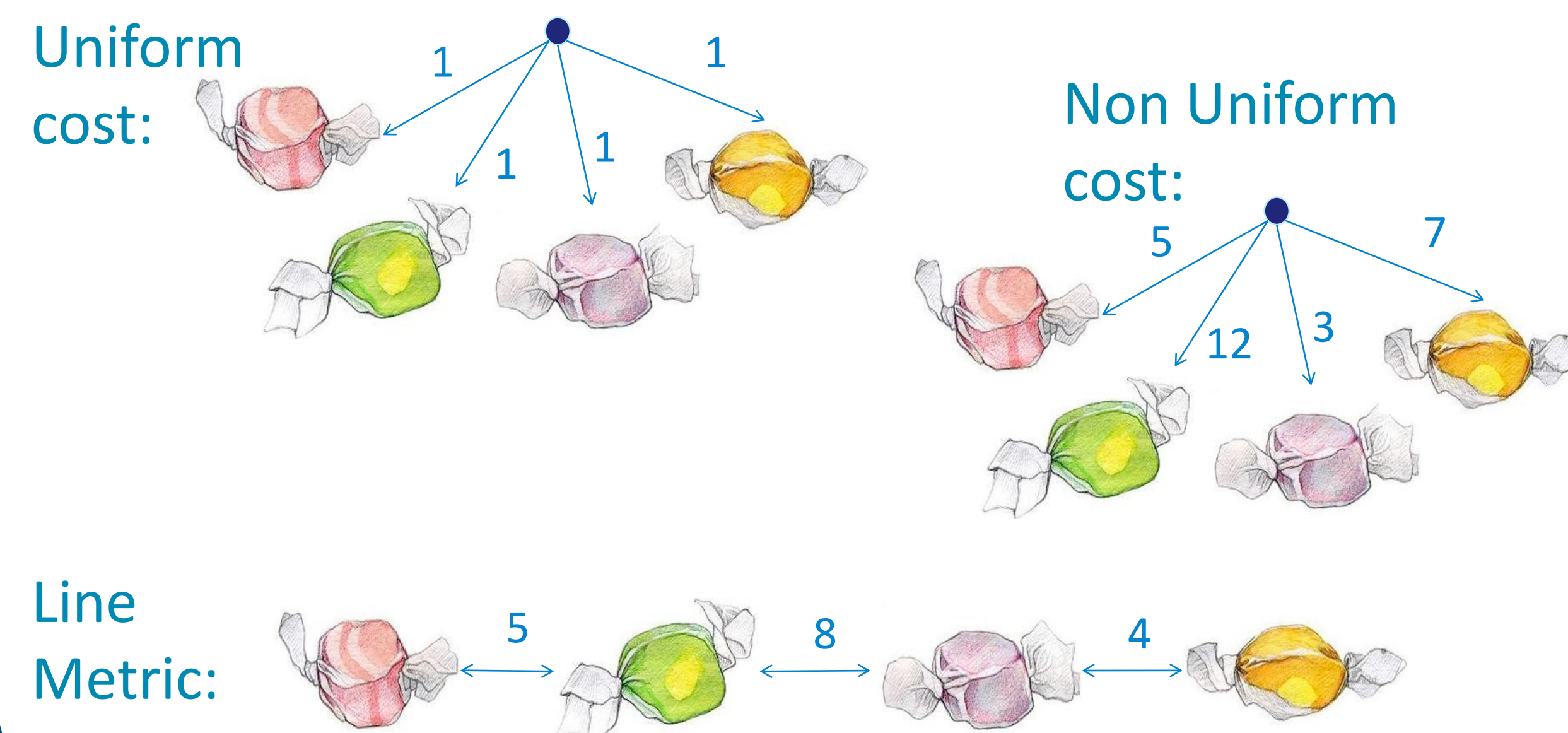


# Reordering Buffer : Improved Algorithm

## The Reordering Buffer Game Rules

- A long sequence of candies  enters a reordering buffer of capacity  $k$ .
- When the buffer is full, the player has to eat a candy from the buffer in order to make room for the next input candy. When no more candies arrive, the player has to eat the remaining candies in the buffer.
- Each time the player changes the color of the candy she eats, she pays a penalty (that depends on the candies).
- The player's goal is to eat all the candies at minimum cost.

## Useful Penalty Metrics



## Online vs. Offline

- Online game: the player needs to decide in each step without knowing what's up ahead.
- Offline game: the player can see the whole sequence of candies.
- The performance of an algorithm is measured by comparing it to an optimal offline solution. This is called the **competitive** ratio or **approximation** ratio of the algorithm.

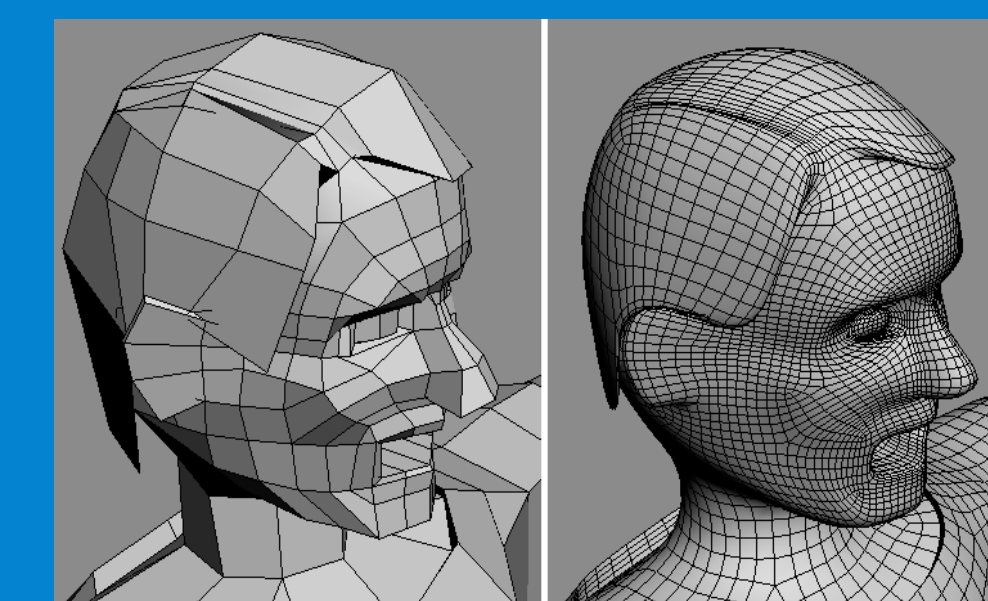
## Motivation

**Painting Cars:** A reordering buffer can be used in a car manufacturing plant between the assembly-line and the paint shop to rearrange the cars waiting to be painted in a way that minimizes the setup costs involved in color changes (Uniform costs).

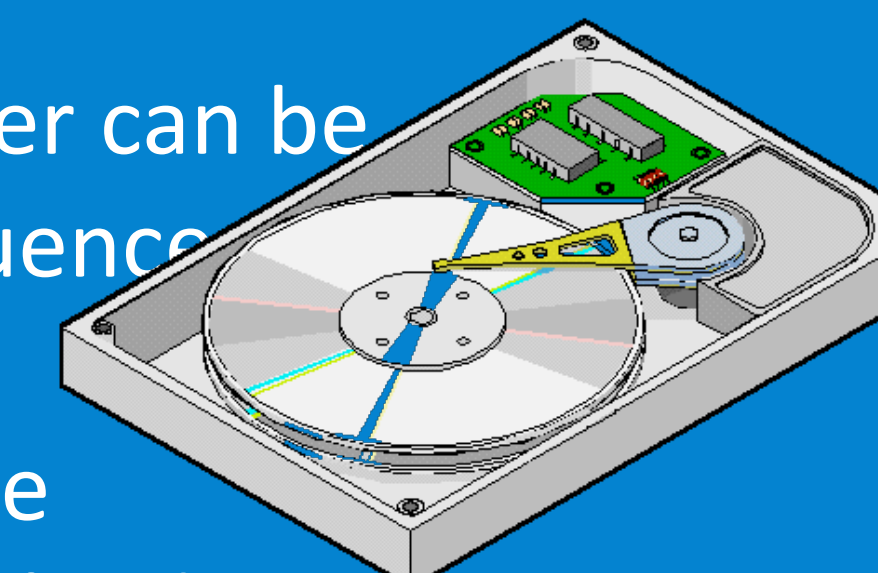
I think we're finished with this one, let's move on to Daddy's Porsche



**Computer Graphics:** A reordering buffer can be included between an application and the graphics hardware to rearrange the incoming sequence of primitives to be rendered such that the cost of the state changes due to switching between different types of primitives is reduced (Non-uniform costs).



**Disk Scheduling :** A reordering buffer can be used to rearrange the incoming sequence of accesses to the hard disk in a way that minimizes the movements of the head to the respective cylinders and by doing so improves the latency of the access. (Line metric)



## Our Online Algorithm (Uniform Metric)

- Choose one color as the active color. Remove at each time step one object of this active color from the reordering buffer while it is still possible.
  - Compute color penalty counters  $P(x)$ , initially set to 0.
  - Choose a new color according to the following rules:  
If there is a color  $c$  such that  $P(c) \geq 1$  we choose  $c$ .  
Otherwise, we choose an arbitrary color  $c$  in the buffer.
  - The counters are updated only in case 2:  
I.  $P(x) \leftarrow P(x) + n(x)/k$   
II.  $P(c) \leftarrow 0$
- $n(x)$  denotes the number of objects of color  $x$  stored in the buffer when we switch to color  $c$ .

## Our algorithm achieves a competitive ratio of

$$\beta = O\left(\frac{\log(k)}{\log\log(k)}\right)$$

The best previous algorithm and analysis gave  $O(\log k)$  competitive ratio

## Analysis

### A relaxation to the problem:

Minimize :  $\sum_{i,j} \sum_{j \leq l(i)-1} x_{i,j} + \#colors$

s.t :  $\sum_{j \geq i} x_{i,j} \geq 1, \sum_{i \leq j} x_{i,j} \leq 1$

$x_{l(i),j} - x_{i,j-1} \geq 0, x \geq 0$

( $l(i)$  is the next item of the same color as  $i$ )

Pay  $x_{i,j}$  if the  $i^{th}$  element is served before seeing  $l(i)$ .

The order between elements of the same color is preserve.

Each element is served at least once and each time step at most one element is served.

### The Dual:

Maximize :  $\sum_{i=1}^N y_i - \sum_{j=k+1}^{k+N} z_j + \#colors$

s.t :  $y_i - z_j \leq u_{i,j+1} - u_{p(i),j}$

$y, z, u \geq 0$

- Using our algorithm, we set:

$$z_j \leftarrow \frac{\#color \text{ changes until step } j}{k}$$

$$y_i \leftarrow z_{j(i)} + \frac{\text{penalty of } i}{\beta}$$

(the algorithm outputs  $i$  in step  $j(i)$ )

- Lemma:  $\exists u$  s.t.  $(y, z, u)$  is a feasible dual solution with value  $DS \geq \frac{1}{\beta} ALG$
- Conclusion:  $ALG \leq \beta \cdot DS \leq \beta \cdot OPT$

By the weak duality theorem.

Presented by: Noa Elgrabli  
Advisor: Yuval Rabani